第7章 第1节 为什么需要 MCP

- 来源: https://class.imooc.com/lesson/5278#mid=104930
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: MCP核心概念
- 课节: 图文 1-1 为什么需要 MCP

1-1 为什么需要 MCP

第7章 第1节 为什么需要 MCP

> [!TIP] 阅读指南 本节从工具共享的难题出发，引出 Function Calling 在工具分发方面的局限，再介绍 MCP 如何解决这些问题。

上一章介绍了 Function Calling，让 AI 能够调用工具、执行实际操作。但写完一个工具之后，真正的问题是：怎样让别人也能用上这个工具?

MCP（Model Context Protocol） 正是为解决这个问题而生。

1.1 工具分享的难题

工具孤岛

假设写了一个很好用的文件操作工具，能让 AI 读取、修改、搜索项目文件。现在，想分享给别人用。

传统方式：把代码发给对方，对方复制到自己项目里，看懂代码、适配自己的 AI 应用。如果更新了工具，对方还得重新复制一遍。这就像没有 npm 的时代，想用一个 JavaScript 库，只能去 GitHub 下载源码、手动复制到项目里、手动管理版本和依赖。有 npm 后，一行命令就解决：

```
npm install lodash

1
```

1

install

AI 工具生态需要的是同样的东西。

1.2 MCP 的诞生

时间线

2024年11月25日，Anthropic（Claude 的开发公司）发布了一个重要的开源协议：Model Context Protocol（MCP）。

虽然这是Anthropic公司定义的私有协议，但随后Anthropic将其开源作为一个开放标准，任何人都可以实现，任何模型都可以支持。

MCP 要解决什么

MCP 的核心目标是建立 AI 工具的生态系统，就像 npm 之于 JavaScript、pip 之于 Python。但盯住单一目标还不够——这个生态要能立住，必须同时解决三个环环相扣的问题：工具怎么让所有人复用、权限怎么统一管控、接入方式怎么标准化。

一个场景，三个痛点

假设要给团队做一个 AI 助手，能查内部文档、查产品数据库、搜最新行业新闻。听起来不复杂——写三个 Function Calling 工具就行。

第一个工具写完，文件操作跑通了。同事觉得好用，问能不能也装一个。把代码发过去，他折腾了一下午才配好环境。过两天工具更新了一个 bug，又得通知所有人重新下载。三个人的团队尚且如此，三十个人的部门呢？

第二个工具连数据库。写完发现权限是个大问题——这个工具能访问所有表，但需求只是查产品表。限制权限的代码散落在几个函数里，改起来提心吊胆，怕一不小心把别的逻辑弄坏。

第三个工具接搜索 API。这时候老板说，市场部用的 AI 客户端和开发部不是同一个平台——一个用 Claude Desktop，一个用 Gemini。工具要写两遍？

三个问题不是孤立的。它们指向同一件事：AI 工具缺一个统一的生态底座。MCP 就是在这个背景下诞生的。

1.3 MCP 如何实现这些目标

工具复用：不再各写各的

现实是每个 AI 应用都在重复造轮子——文件操作自己写一套，数据库查询自己写一套，网络搜索自己写一套。三个应用就是三套重复代码。

MCP 的做法更像 npm：同一个工具，一个人写好发布，其他人一行命令就能装。不用复制代码、不用配环境、不用操心版本更新。

实际效果



而且 MCP Server 往往是社区维护的——经过了大量测试，比自己从零写的更可靠。

统一权限：配置即安全边界

传统 Function Calling 的权限管理藏在代码里。每个工具自己判断该不该给权限，规则散落在各处，不统一也看不见。比如想让文件操作工具只能碰项目目录、别碰个人文件——这个约束写在哪个函数的哪一行里，只有最初写的人知道。

MCP 把权限控制提到了配置层。启动 Server 时就明确声明它能访问的目录、端口、资源范围——在 mcpServers 配置里白纸黑字写清楚，随时查看、随时修改。

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/username/Projects/my-app"  // 只能访问这个目录
      ]
    }
  }
}
```

文件操作 Server 被限制在 /Users/username/Projects/my-app，其他地方碰不到。权限不再是散落在代码里的零散判断，而是一份可审计的配置。

标准接入：一个协议通吃

假设要开发一个 AI 助手，需要文件操作、数据库查询、网络搜索、GitHub 集成。没有 MCP 的话，得分别对接四套 SDK、四种认证方式、四种错误处理逻辑——每加一种能力就多一层复杂度。

MCP 定义了一套统一的 Client-Server 协议。接入新工具不挑语言、不挑平台，只需要一行命令：

```
npm install -g @modelcontextprotocol/server-filesystem
npm install -g @modelcontextprotocol/server-postgres
npm install -g @modelcontextprotocol/server-brave-search
npm install -g @modelcontextprotocol/server-github
```

四种能力、同一套通信协议、相同的调用方式。AI 应用只需要实现一个 MCP Client，剩下的能力通过 Server 生态按需扩展——不再需要为每个新工具单独写集成代码。

1.4 MCP 的另一面

MCP 虽然号称开放标准，但 SDK 和官方文档里到处都是 Claude 的痕迹。这不是巧合。

MCP 是由 Anthropic（Claude 的开发公司）在 2024 年 11 月发布的。严格来说：

```
+--------------------------------+
| [v] 开放（代码公开、文档公开）
| [v] 标准（有明确的协议规范）
| [x] 但不中立（Anthropic 主导）
+--------------------------------+
```

对比

传统的开放标准（如 HTTP、JSON）由中立组织（W3C、IETF）制定，多方参与，商业中立。

但 MCP 更像这些标准：

```
+----------------------------------------+
| TypeScript：微软创造，开源但微软主导
| Kubernetes：Google 创造，CNCF 管理
| Rust：Mozilla 创造，Foundation 管理
+----------------------------------------+
```

MCP 目前的情况是 Anthropic 主导，虽然完全开源。

为什么会这样？这反映了 MCP 的诞生背景：

```
+--------------------------------+
| MCP 的设计初衷是为 Claude 赋能
|     |
|     v
| 发现这个能力是通用的
|     |
|     v
| 开源成开放标准
|     |
|     v
| 但保留了 Claude 特有特性
+--------------------------------+
```

比如，MCP 中的某些枚举值（如 tool_use 的 stop_reason）直接来自 Claude 的 API 设计。

1.5 下一节预告

理解了为什么需要 MCP。下一节会系统拆解 MCP 的核心概念和整体架构，并和传统的 Function Calling 进行对比。

1.6 ■ 学点英语

中文	English	音标	说明
模型上下文协议	Model Context Protocol (MCP)	/ˈmɑːdl ˈkɑːntekst ˈproʊtəkɑːl/	统一AI应用与外部工具交互的标准化协议
工具孤岛	Tool Isolation	/tuːl ˌaɪsəˈleɪʃn/	工具与应用高度耦合，无法在多个AI应用间共享的问题
标准化协议	Standardized Protocol	/ˈstændərdaɪzd ˈproʊtəkɑːl/	定义统一规范使不同组件可互操作的协议标准
语言服务器协议	Language Server Protocol (LSP)	/ˈlæŋgwɪdʒ ˈsɜːrvər ˈproʊtəkɑːl/	标准化编辑器与语言服务之间通信的协议，MCP的灵感来源